

Lab 3 Advanced: FPGA Seven-Segment Display & Buttons

Submission Due Dates:

Demo: 2026/10/13 17:20

Source Code: 2026/10/13 18:30

Report: 2026/10/18 23:59

Objective

This exercise will help you gain hands-on experience with FPGA seven-segment display and buttons design with FSM in Verilog. In this lab, you will design an adder and a marquee.

Action Items 1: 2-digit simple adder

1. lab3_advanced_1.sv

In this lab, you must design a 2-digit adder based on a finite-state machine (FSM). Then you can control this adder using switches and buttons, as described below.

a. I/O list

I/O	Connected to	Definition
clk	W5	Clock signal with the frequency of 100MHz
rst	SW15	Asynchronous active-high reset signal
mode_state	LD1~0	Show the current state
number	SW7~SW0	Represent a 2-digit number in BCD (8421-BCD)
set	BTNC	Set number and go to next state
DISPLAY[7:0]		Show number
DIGIT[3:0]		Select active digit

b. FSM Specification

The FSM should consist of four states: **INITIAL**, **SET1**, **SET2**, and **RESULT**.

- At any time, **rst == 1** asynchronously resets the FSM to the **INITIAL** state.
- When **rst == 0**, the FSM resumes normal operation.

✓ INITIAL state

- LEDs (**LD1~LD0**) are **off**.

LD1oo LD0

(● : LED on, ○ : LED off)

- All four seven-segment displays shall be off, as shown in the figure below.



- During normal operation, the FSM transitions from the **INITIAL** state to the **SET1** state at the next rising edge of clk.

✓ **SET1** state

- Entered from the **INITIAL** state.
- LEDs (**LD1~LD0**) are represented as below.

LD1○● LD0

(● : LED on, ○ : LED off)

- The left two digits of the seven-segment display blink simultaneously, toggling between visible and blank **every 0.5 seconds**. The displayed decimal value is determined by SW7–SW0, where SW7–SW4 correspond to the left digit and SW3–SW0 correspond to the right digit.

EX: Here, we take $67 + 87 = 154$ as an example.



- Press BTNC to enter SET2 state.
- Note:
 - When the value of SW7–SW0 changes, the displayed digits may be updated either in the current clock cycle or in the next clock cycle. Either behavior is acceptable.
 - You may assume that both BCD digits are valid. Values from 4'hA to 4'hF will not be tested. (BCD encoding here refers to the "[8421](#)" version.)
 - Since the other addend has not yet been set, the **right two digits of the seven-segment display should show 00 in the SET1 state.**

✓ SET2 state

- Entered from the **SET1** state when **BTNC** is pressed.
- LEDs (**LD1~LD0**) are represented as below.
LD1●○ LD0
(● : LED on, ○ : LED off)
- The right two digits of the 7-segment display blink simultaneously, toggling between visible and blank **every 0.5 seconds**. The displayed decimal value is determined by SW7–SW0, where SW7–SW4 correspond to the left digit and SW3–SW0 correspond to the right digit. (same requirement as **SET1**)
- At the same time, the **two digits on the left should continue to display the numbers set in the previous SET1 state**.

EX:



- Press BTNC to enter **RESULT** state.
- Note (as mentioned above):
 - When the value of SW7–SW0 changes, the displayed digits may be updated either in the current clock cycle or in the next clock cycle. Either behavior is acceptable.
 - You may assume that both BCD digits are valid. Values from 4'hA to 4'hF will not be tested.

✓ RESULT state

- Entered from the **SET2** state when **BTNC** is pressed.
- LEDs (**LD1~LD0**) are represented as below.
LD1●● LD0
(● : LED on, ○ : LED off)
- In the **RESULT** state, the 7-segment display shows the correct sum of the two input numbers.
- Press **BTNC** to enter **SET1** state and start another addition operation.

EX:



- Note:

We only consider addition of two non-negative integers, so the smallest sum is $0 + 0 = 0$, and the largest sum is $99 + 99 = 198$. Overflow is not considered.

You must use the following template for your design:

```
module lab3_advanced_1 (  
    input  logic clk,  
    input  logic rst,  
    input  logic [7:0] number,  
    input  logic set,  
    output logic [1:0] mode_state,  
    output logic [7:0] DISPLAY,  
    output logic [3:0] DIGIT  
);  
    // add your design here  
endmodule
```

Hint

1. Before implementing the FSM block, it is suggested to **draw the FSM diagram** first.

In this question, the FSM process should be **INITIAL** → **SET1** → **SET2** → **RESULT**.

Action Items 2: Marquee

2. lab3_advanced_2.sv

In this lab, you must design a marquee based on a finite-state machine (FSM). Then you can control this marquee using switches and buttons, as described below.

a. I/O list

I/O	Connected to	Definition
clk	W5	Clock signal with the frequency of 100MHz
rst	SW15	Asynchronous active-high reset signal
speed	SW0	Control velocity of Marquee
right	BTNR	Set Marquee moving direction to right
left	BTNL	Set Marquee moving direction to left
pause	BTNC	Control Marquee stop/resume
mode_state	LD1~LD0	Show FSM state
dir	LD2	Indicate the current marquee moving direction (OFF: left, ON: right)
DISPLAY[7:0]		Show number
DIGIT[3:0]		Select active digit

b. FSM Specification

The FSM should consist of three states: **INITIAL**, **RUN**, and **PAUSE**.

- At any time, **rst == 1** asynchronously resets the FSM to the **INITIAL** state.
- When **rst == 0**, the FSM resumes normal operation.

✓ INITIAL state

- LEDs (**LD2~LD0**) are **off**.

LD2 o o o LD0

(● : LED on, o : LED off)

- The internal direction variable shall be initialized to left.
- All four digits of the seven-segment display shall be off.



- During normal operation, the FSM transitions from the **INITIAL** state to the **RUN** state at the next rising edge of **clk**.

✓ RUN state

- The marquee should start from "**nthu**" after entering this state and continuously scroll left (before any buttons are pressed) the string

"nthuEECS2026" in a circular manner (each letter and number pattern refers to the figure corresponding to "nthuEECS2026").



Reference pattern



Begin with "nthu"

- The marquee repeats indefinitely.
- LEDs (LD2~LD0) are represented as below at first.

LD2o

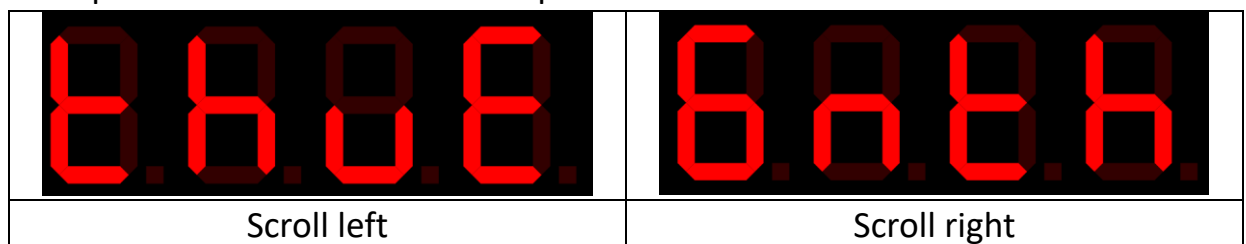
LD1o● LD0

(● : LED on, o : LED off)

- Pressing **BTNC** enters the **PAUSE** state.

● Direction

- Then LD2 indicates the current scrolling direction (initially **left** from the **INITIAL** state).
 - LD2 = OFF: scrolling left
 - LD2 = ON: scrolling right
- Pressing **BTNL** changes the scrolling direction to **left**.
- Pressing **BTNR** changes the scrolling direction to **right**.
- The scrolling direction may take effect immediately or change in the next update. Either behavior is acceptable.



● Speed

- The scrolling speed is determined by SW0.
 - SW0 = 0: shift one position every 1 second
 - SW0 = 1: shift one position every 0.5 seconds
- When SW0 changes, the scrolling speed may be updated either in the current clock cycle or in the next clock cycle. Either behavior is acceptable.

✓ PAUSE state

- entered when **BTNC** is pressed during the **RUN** state.
- LEDs (LD2~LD0) are represented as below.

LD2 = Current Direction

LD1●○ LD0

(● : LED on, ○ : LED off)
- LD2 continues to indicate the current scrolling direction.
- **The seven-segment display stops scrolling and maintains the currently displayed characters.** (The displayed characters may remain the current ones or be updated to the next scrolling position (at the moment BTNC is pressed). Either behavior is acceptable.)
- While paused, the current display position and scrolling direction shall be preserved, including
 - the current display position. (7-seg. display)
 - the current scrolling direction. (LD2)
- Pressing BTNL or BTNR during the **PAUSE** state shall **have no effect**.
- Changing **SW0** during the **PAUSE** state takes effect immediately after returning to the RUN state.
- Pressing **BTNC** again returns to the **RUN** state and resumes scrolling from the preserved position.

All preserved states are reset only when **rst (SW15)** is asserted.

You must use the following template for your design:

```
module lab3_advanced_2(  
    input  logic clk,  
    input  logic rst,  
    input  logic speed,  
    input  logic left,  
    input  logic right,  
    input  logic pause,  
    output logic [1:0] mode_state,  
    output logic dir,  
    output logic [7:0] DISPLAY,  
    output logic [3:0] DIGIT  
);  
  
    // add your design here  
endmodule
```

3. Questions and Discussion

- A. Why are debounce and one-pulse modules required?
- B. Can the decimal point on the bottom-right of the seven-segment display be controlled? Which pin should it be assigned to in the .xdc file?
- C. How do you implement a signal that is triggered only when a button is double-clicked quickly, how would you implement it in Verilog?

Attention

- ✓ If you create several modules, merge them into a single Verilog file.
- ✓ Feel free to ask questions about the specification on the EECLASS forum.
- ✓ DO NOT copy-and-paste code segments from PDFs (hidden characters may cause hard-to-debug syntax errors).
- ✓ No special consideration needs to be given to exceptions not mentioned in this question, such as: pressing BTNL and BTNR simultaneously, or changing direction instantaneously when returning from PAUSE to RUN (pressing BTNC and BTNR/L simultaneously).
- ✓ You must make sure that the buttons have been **debounced and one-pulsed**. Points will be deducted if the button is not debounced and one-pulsed.

Submission and Demo

- ✓ You should submit **two** source files, **lab3_advanced_1.sv** and **lab3_advanced_2.sv** . **DO NOT** hand in any compressed ZIP files, which will be considered an incorrect format.
- ✓ You should also hand in your report as **lab3_report_StudentID.pdf** (i.e., lab3_report_110062320.pdf). Please combine the reports from the two questions into one file (for report).
- ✓ You should be able to answer questions about this lab from TA during the demo.
- ✓ You need to prepare the bitstream files before the lab demo to make the demo process smooth.

Grading Breakdown for Demo

Functionality	Weight
lab3_advanced_1	60%
INITIAL state	5%
mode_state(LED 0~1)	20%(each state 5%)
switches & 7-seg. display	30%(SET1: 15%, SET2: 15%)*
RESULT state	5%

* switches & 7-seg. display: 10%, 7-seg. display blinking: 5%

Functionality	Weight
lab3_advanced_2	40%
INITIAL state	5%
RUN state(7-seg. display)	10%
RUN state(BTNL & BTNR)	10% (each button is 5%)
RUN state (speed)	5%
PAUSE state	10%